

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software
Aplicaciones Distribuidas (ISR-701)

Informe de Revisión Técnica Cruzada

del Código del Proyecto Fin de Curso

Taller FORM-TL-05 — Unidad 5
Principios SOLID, patrones de diseño, separación de capas,
manejo de excepciones distribuidas y observabilidad

Equipo revisor: BCEL
Sistema revisado: SCLI — Sistema de Control de Laboratorios e Infraestructura
Equipo autor del PFC: FUVV
Fecha de auditoría: 11 de agosto de 2026

Integrantes del equipo revisor:

Castro López Pedro Leonardo	Líder / Editor / Revisión Bloques C, D, E
Emanuel Pino Juliana Romina	Revisión Bloques A y B (SOLID y Patrones)
Villamarín Cuenca Iván Andrés	Integrante
Vinueza Sánchez Harold Nicolás	Integrante

1. Objeto auditado

La revisión se realizó sobre un commit específico del repositorio del equipo FUVV, fijado antes de cualquier análisis para garantizar la reproducibilidad de los hallazgos.

Repositorio revisado	https://github.com/iavillamarin98-pred/Proyecto_Fin_de_C
Rama	main
Commit auditado (SHA completo)	49ec17dfd42835fe5dfbd75249a5e8a51941ef4a
SHA corto	49ec17d
Fecha del commit	2026-07-29 23:40:45 (UTC-05)
Autor del commit	Freddy Vladimir Farinango Guandinango
Archivos versionados	414
Fecha y hora de la auditoría	2026-08-11

2. Fase 1: Congelación del objeto revisado

Antes de iniciar cualquier análisis se ejecutaron los comandos exigidos por la guía del taller para dejar registrada la versión exacta del código sobre la que se produce toda la evidencia posterior. La salida obtenida fue la siguiente:

```
$ git log -1 --format="%H | %aI | %an"
49ec17dfd42835fe5dfbd75249a5e8a51941ef4a | 2026-07-29T23:40:45-05:00 |
Freddy Vladimir Farinango Guandinango
```

```
$ git rev-parse --short HEAD
49ec17d
```

```
$ git ls-files | wc -l
414
```

El sistema levanta correctamente mediante `docker compose up -d` conforme al `README.md` del repositorio auditado. No se detectaron desviaciones que impidieran la revisión.

3. Fase 2: Mapa de arquitectura reconstruido

El sistema auditado, denominado **SCLI** (*Sistema de Control de Laboratorios e Infraestructura*) por sus autores del equipo FUVV, está implementado como un conjunto de microservicios Spring Boot en Java, coordinados mediante Docker Compose y con persistencia en instancias independientes de CockroachDB. El mapa siguiente se reconstruyó leyendo el código fuente y el archivo `docker-compose.yml`, no la documentación.

3.1 Componentes del sistema

Componente	Puerto	Base de datos	Responsabilidad
scli-front	—	—	Interfaz de usuario
api-gateway	8080	—	Enrutamiento (Spring Cloud Gateway MV)
auth-service	8081	cockroach-auth	Autenticación y emisión de JWT
usuarios-service	8082	cockroach-usuarios	Gestión de perfiles
academico-laboratorios-service	8083	cockroach-academico	Datos académicos y laboratorios
reservas-solicitudes-service	8084	crdb-e3 (3 nodos)	Reservas y solicitudes

3.2 Flujo de dependencias entre servicios

- El frontend consume el API Gateway; el Gateway reenvía a los servicios internos por HTTP.
- `auth-service` depende de `usuarios-service` para recuperar el perfil del usuario autenticado.
- `reservas-solicitudes-service` depende de `academico-laboratorios-service` (verificación de materia, periodo lectivo y laboratorio) y de `usuarios-service` (verificación de perfil del solicitante).
- Cada microservicio posee su propia base de datos CockroachDB, lo que respeta el patrón *Database per Service*. El módulo de reservas usa además un clúster CockroachDB de tres nodos como evidencia de replicación distribuida.

3.3 Estructura interna común

Todos los microservicios de dominio siguen la misma organización de paquetes Java, con separación explícita en capas:

- `controller/` — Endpoints REST (capa de presentación).
- `service/` y `service/impl/` — Lógica de aplicación.
- `repository/` — Acceso a datos (Spring Data JPA).
- `entity/` — Entidades JPA (modelo de dominio persistente).
- `dto/` (`request/`, `response/`) — Objetos de transferencia hacia la interfaz pública.
- `exception/` — Excepciones de negocio y manejador global.
- `specification/` — Consultas dinámicas.
- `mapper/` (solo reservas) — Conversión entidad $\rightarrow$ DTO.
- `client/` y `config/` (reservas y auth) — Clientes REST hacia otros servicios y configuración de `RestClient`.
- `security/` — Filtros y proveedores JWT.

Esta estructura evidencia una separación de capas real (no meramente por nombre de carpeta), lo que se profundiza en el análisis del Bloque C.

4. Fase 3: Tabla consolidada de hallazgos

La siguiente tabla resume los nueve hallazgos verificados sobre el código de FUVV en el commit 49ec17d. Cada hallazgo se convirtió en un *issue* en el repositorio revisado con la plantilla obligatoria de la guía del taller.

ID	Ítem	Hallazgo	Sev.	Issue
H01	B1/B4	API Gateway con configuración duplicada e inconsistente (rutas en Java y en YAML a la vez).	Crítica	#1
H02	C5	URLs <code>http://localhost:8081/8082/8083</code> incrustadas en el Gateway; inoperantes bajo Docker.	Crítica	#2
H03	B4	<code>reservas-solicitudes-service</code> no está registrado en el Gateway.	Mayor	#3
H04	D3	Reintentos sin <i>backoff</i> exponencial ni límite temporal.	Mayor	#4
H05	D5	Mensajes internos de excepción filtrados al cliente.	Mayor	#5
H06	D5	<code>IllegalArgumentException</code> mapeada a HTTP 404.	Menor	#6
H07	E2/E3	Ausencia de identificador de correlación entre servicios.	Mayor	#7
H08	D1/E5	Captura silenciosa de excepciones en el bucle de reintentos.	Mayor	#8
H09	A1	Violación de SRP y duplicación en <code>AuthService.login/refreshar</code> .	Menor	#9

Distribución de severidad: 2 Críticas, 5 Mayores, 2 Menores. Cobertura de los cinco bloques del checklist: A, B, C, D y E.

5. Análisis por bloque

5.1 Bloque A — Principios SOLID

SRP (Cumple Parcialmente). En `auth-service/.../service/AuthService.java:43-109`, el método `login` concentra: autenticación local, llamada remota al servicio de usuarios, validación de perfil activo, generación de dos tokens JWT, transformación de *authorities* en roles y permisos, y ensamblaje del DTO de respuesta. El método `refreshar` (líneas 111-165) duplica casi por completo el bloque final de construcción del `AuthUserResponse`.

ISP (Cumple). Las interfaces de servicio están segregadas por dominio (por ejemplo `PerfilService`, `EstudianteService`, `DocenteService`) y no se detectaron implementaciones vacías.

DIP (Cumple). La inversión de dependencias se aplica de forma sistemática. Todos los componentes revisados reciben sus colaboradores por constructor: `AuthService` (líneas 31-41), `SolicitudReservaController` (líneas 44-46) y `AcademicoLaboratoriosClient` (líneas 22-27). La capa de dominio no importa el driver de base de datos ni el cliente HTTP concreto.

5.2 Bloque B — Patrones de diseño

Patrones implementados (Cumple). Se identificaron tres patrones efectivamente aplicados:

- **Repository** en el paquete `repository/` de cada servicio, apoyado en Spring Data JPA.
- **Specification** (variante del patrón *Query Object*) en `academico-laboratorios-service/.../specificaciones/` para consultas dinámicas y componibles.
- **DTO Mapper** en `reservas-solicitudes-service/.../mapper/` (`ReservaMapper`, `SolicitudReservaMapper`) que aísla las entidades JPA de la interfaz pública.

Patrones distribuidos (No Cumple). Existe un módulo `api-gateway` que implementa el patrón *API Gateway*, pero la implementación presenta tres defectos verificados:

1. En `api-gateway/.../routes/GatewayRoutes.java:14-33` se declaran rutas por código Java para `auth-service` y `usuarios-service`.
2. En `api-gateway/src/main/resources/application.yml:8-25` se declaran rutas por configuración YAML para `auth`, `usuarios` y `académico`, con *predicados distintos* a los del código Java.
3. El servicio `reservas-solicitudes-service:8084` no aparece en ninguno de los dos ficheros, por lo que no es enrutable desde el Gateway.

Circuit Breaker (No Cumple). No se detectó implementación de disyuntor (Resilience4j o equivalente). Ante una caída sostenida de `académico-laboratorios`, la respuesta del sistema depende únicamente de tiempos de espera y reintentos ciegos.

5.3 Bloque C — Separación de capas

C1-C4 (Cumple). La estructura de paquetes reproduce fielmente la separación por responsabilidades. Las entidades JPA no se exponen en los endpoints REST: existe una capa `dto/` bien delimitada. La lógica de negocio reside en las clases `*ServiceImpl` y no en los controladores, que se limitan a delegar.

C5 — Configuración externalizada (No Cumple). Aunque credenciales de base de datos y claves JWT sí se externalizan mediante variables de entorno en `docker-compose.yml`, en el Gateway las URLs de destino están *incrustadas en el código fuente* (`GatewayRoutes.java:20,28` y `application.yml:13,18,23`) como `http://localhost:8081/8082/8083`. Dentro de la red Docker `sccli-network`, los servicios se resuelven por nombre de contenedor (`auth-service:8081`); por tanto, esta configuración no es portable.

5.4 Bloque D — Manejo de excepciones distribuidas

D1 — Captura silenciosa (No Cumple). En `reservas-solicitudes-service/.../client/AcademicoOL` el método `executeWithReadRetries` captura `ResourceAccessException` y `HttpServerErrorException` pero no registra las excepciones intermedias antes de reintentar. Un fallo transitorio no deja rastro.

D2 — Timeouts explícitos (Cumple). `RestClientConfig.java:38-53` configura `connectTimeout` y `readTimeout` en el `SimpleClientHttpRequestFactory` a partir de las propiedades `app.http.connect-timeout` y `app.http.read-timeout-ms`.

D3 — Reintentos acotados con espera creciente (No Cumple). El bucle de reintentos en `AcademicoLaboratoriosClient.java:60-72` es un `while(true)` sin `Thread.sleep` ni *backoff exponencial*. Aunque el número de intentos está limitado por `maxReadRetries`, los reintentos se disparan de forma inmediata, agravando la carga sobre el servicio remoto y sin darle tiempo a recuperarse.

D4 — Respuesta de degradación (No Cumple). No se observa valor de reserva ni respuesta de degradación cuando `académico-laboratorios` está indisponible. El fallo se propaga y se transforma en HTTP 503 al usuario.

D5 — Errores uniformes al cliente (Cumple Parcialmente). `GlobalExceptionHandler.java` unifica el formato de respuesta en un `ApiError` con `timestamp`, `status`, `error`, `message` y `path`. Sin embargo:

- El manejador genérico `manejarExcepcion(Exception)` (líneas 80-84) devuelve `exception.getMessage()` sin filtrar, y el manejador de `DataIntegrityViolationException` (líneas 56-60) hace lo mismo. Esto puede exponer nombres de tablas o restricciones internas al cliente.
- `IllegalArgumentException` se mapea a HTTP 404 (líneas 23-27), cuando su semántica natural es 400 Bad Request.

5.5 Bloque E — Logging y trazabilidad

E1 (Cumple parcialmente). El proyecto utiliza SLF4J/Logback por defecto (Spring Boot). No se detectó uso indebido de `System.out` en las clases revisadas.

E2 — Identificador de correlación (No Cumple). En `RestClientConfig.java:38-53`, el único encabezado propagado en las llamadas entre servicios es `X-Internal-API-Key`. No se propaga `X-Correlation-Id` ni `traceparent` (W3C Trace Context). Tampoco se detectó configuración de MDC (*Mapped Diagnostic Context*).

E3 — Registro estructurado (No Cumple). La configuración por defecto produce registros en texto plano. No se detectó `logstash-logback-encoder` u otro *appender* JSON que permita filtrar por servicio, operación o identificador de correlación.

E4 (Cumple). No se observó registro de contraseñas ni tokens en las clases revisadas.

E5 — Huella de eventos críticos (No Cumple). Las creaciones y transiciones de estado de solicitudes y reservas no dejan registro auditable explícito en los servicios revisados.

6. Fase 4: Issues abiertos en el repositorio revisado

Se abrieron nueve *issues* en el repositorio de FUVV (https://github.com/iavillamarin98-pred/Proyecto_Fin_de_Curso_scli/issues), uno por cada hallazgo, con etiquetas de bloque y severidad, responsable nominal del equipo autor y fecha de resolución anterior al cierre de la Semana 17.

Nº	Título	Etiquetas	Fecha
#1	Unificar configuración de rutas del API Gateway	patrones, critica	2026-08-18
#2	Reemplazar URLs <code>localhost</code> por nombres Docker	capas, critica	2026-08-18
#3	Registrar <code>reservas-solicitudes</code> en el Gateway	patrones, mayor	2026-08-18
#4	Introducir <i>backoff</i> exponencial en los reintentos	excepciones, mayor	2026-08-18
#5	Sanear mensajes de error expuestos al cliente	excepciones, mayor	2026-08-18
#6	Corregir mapeo de <code>IllegalArgumentException</code> a 400	excepciones, menor	2026-08-18
#7	Propagar identificador de correlación entre servicios	observabilidad, mayor	2026-08-18
#8	Registrar excepciones en el bucle de reintentos	observabilidad, mayor	2026-08-18
#9	Extraer construcción del <code>AuthUserResponse</code>	solid, menor	2026-08-18

7. Preguntas de análisis obligatorias

1. ¿Qué violación de SOLID encarece más el cambio para la Entrega 4? La violación de SRP en `AuthService` (H09). La duplicación de la construcción del `AuthUserResponse` en `login` y `refrescar` implica que cualquier ajuste al modelo de identidad (nuevos campos de perfil, nuevas políticas de expiración) exigirá editar dos métodos con riesgo alto de divergencia silenciosa.

2. ¿Qué patrón introduciría que hoy no está? Un *Circuit Breaker* sobre `AcademicoLaboratoriosClient` y `UsuariosClient` usando `Resilience4j`. Actualmente los reintentos son ciegos y sin freno; un disyuntor interrumpiría las llamadas ante indisponibilidad sostenida y permitiría degradar la funcionalidad de forma controlada. Aumentar el *timeout* no basta porque no protege al servicio remoto de la avalancha de reintentos ni permite pasar a modo degradado.

3. Si académico-laboratorios deja de responder durante 30 segundos, ¿qué ocurre? Cada llamada de `reservas` espera `readTimeoutMs` y arroja `ResourceAccessException`. El bucle `while(true)` de `executeWithReadRetries` reintenta inmediatamente hasta `maxReadRetries` veces. El bloqueo total por hilo puede alcanzar $(readTimeoutMs) \times (maxReadRetries + 1)$ milisegundos, tras lo cual `GlobalExceptionHandler` devuelve HTTP 503 al usuario con el `getMessage()` de la excepción. No hay degradación ni respuesta de reserva.

4. **¿Se puede reconstruir la traza de una operación entre dos servicios?** No. Ninguna cabecera de correlación se propaga y los *logs* no incluyen un identificador común. Al crear una solicitud de reserva, `reservas-solicitudes-service` llama a `académico-laboratorios` para verificar materia y periodo lectivo; en los *logs* de ambos servicios aparecen entradas independientes sin ningún campo que las relacione.

5. **¿Qué aprendemos para nuestro propio PFC (BCEL)?** Antes de la Entrega 4 adoptaremos: (i) un filtro de correlación en el borde del sistema que genere `X-Correlation-Id` y lo propague vía MDC y cabeceras salientes; (ii) configuración del *gateway* en una única fuente declarativa; (iii) Resilience4j para disyuntor y *backoff* exponencial; (iv) un `GlobalExceptionHandler` que nunca devuelva `getMessage()` de excepciones internas al cliente.

8. Conclusiones y recomendación priorizada

El sistema SCLI presenta una arquitectura de microservicios con separación de capas real y patrones estructurales bien aplicados (Repository, Specification, DTO Mapper). Las debilidades se concentran en los ejes propiamente distribuidos: el API Gateway está desconfigurado y no cubre a todos los servicios; el manejo de fallos parciales es prematuro (reintentos sin *backoff*, sin disyuntor y sin degradación); y la observabilidad está esencialmente ausente por la falta de identificador de correlación propagado entre servicios.

9. Declaración de uso de inteligencia artificial generativa

El equipo revisor BCEL declara haber utilizado un asistente de inteligencia artificial únicamente como apoyo puntual para revisión ortográfica y sugerencias de estilo durante la redacción final del documento. El análisis del código, la identificación de hallazgos, la elaboración de los *issues* y la selección de referencias fueron realizados directamente por los integrantes del equipo. La responsabilidad intelectual del contenido corresponde íntegramente a los autores del informe.

Referencias

- [1] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, (San Francisco, CA, USA), pp. 712–721, 2013. doi: 10.1109/ICSE.2013.6606617.
- [2] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, “Modern code review: A case study at google,” in *Proc. 40th Int. Conf. Softw. Eng. Softw. Eng. Pract. (ICSE-SEIP)*, (Gothenburg, Sweden), pp. 181–190, 2018. doi: 10.1145/3183519.3183525.
- [3] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017. <https://www.pearson.com/en-us/subject-catalog/p/clean-architecture-a-craftsman-s-guide-to-software-structure-and-design/P200000009528>.
- [4] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018. <https://www.manning.com/books/microservices-patterns>.
- [5] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2nd ed., 2021. <https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>.
- [6] D. Taibi and V. Lenarduzzi, “On the definition of microservice bad smells,” *IEEE Software*, vol. 35, no. 3, pp. 56–62, 2018. doi: 10.1109/MS.2018.2141031.

- [7] International Organization for Standardization, “Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model.” ISO, Geneva, Switzerland, 2023. <https://www.iso.org/standard/78176.html>.